

Bamyon - Complete Build Documentation

Bamyon — Complete Build Documentation

A Business-in-a-Box SaaS for Nigerian SMEs, built on Next.js 14 + Firebase, with no backend server of its own.

This document explains everything about how Bamyon is built: the architecture, the complete Firestore data model, how every third-party service is wired in, and a step-by-step guide to recreating the entire project from an empty folder. It's written for a developer picking this project up cold — either you, in six months, or someone else entirely.

1. What Bamyon is

Bamyon is a multi-tenant SaaS dashboard for small Nigerian businesses. Each business owner signs up, gets their own dashboard, logs sales and purchases, tracks debtors, generates PDF receipts, and can add staff logins with page-by-page permissions. A separate admin console (run by Bamyon, not the business owners) manages plan payments, wallet top-ups, withdrawals, and support messages.

Three separate interfaces share one codebase:

Interface	Route prefix	Who uses it
Public marketing site	/	Anyone
Owner dashboard	/dashboard/*	Business owners
Staff dashboard	/staff/*	Staff accounts (permission-limited)
Admin console	/admin/*	Bamyon's own team
Public storefront	/c/[businessId]	A business's customers (no login)

2. Tech stack

Layer	Choice	Why
Framework	Next.js 14 (App Router)	File-based routing across 4 distinct interfaces, API routes for the handful of things that need a real server-side secret
Hosting	Vercel	Zero-config Next.js deploys, serverless API routes included
Database	Firestore (Firebase)	Realtime listeners fit a live dashboard; document model fits per-business data isolation
Auth	Firebase Authentication (Email/Password)	Free, simple, no separate user table to maintain
Styling	Tailwind CSS	Fast iteration, no design system to build from scratch
Image hosting	Cloudinary	Free tier, unsigned upload presets mean no backend upload handler needed
PDF generation	jsPDF + jspdf-autotable	Client-side receipt/invoice generation, no server round-trip

AI (text)	Pollinations.ai (free, keyless)	Message rewriting for admin replies and CRM campaigns
AI (vision)	Pollinations.ai OpenAI-compatible endpoint	Receipt/invoice photo → structured line items
AI (image generation)	Pollinations.ai	Free logo generation
Transactional email	Resend	Premium Pro CRM email campaigns — the one feature needing a real server secret
Automation	n8n (external, self-hosted or cloud)	SMS/WhatsApp review-request messages, triggered by a webhook Bamyon calls

Deliberately not used: Firebase Admin SDK / service account. Every write that would normally go through a trusted server happens directly from the browser instead, secured by Firestore Security Rules. Section 6 explains why this is safe and where its limits are.

3. Complete Firestore data model

This is the actual shape of the database. Every collection below, with every field.

`users/{uid}`

One document per person who can log in — owner, staff, or admin. Document ID is the Firebase Auth UID.

```
{
  uid: string,
  role: "owner" | "staff" | "admin",
  businessId: string | null, // null only for admin accounts
  email: string,
  name: string,
  permissions?: { // present only when role === "staff"
    sales: boolean,
    purchases: boolean,
    inventory: boolean,
    debtors: boolean,
    receipts: boolean,
    invoices: boolean,
    catalogue: boolean,
    crm: boolean,
    orders: boolean,
    reviews: boolean,
    analytics: boolean,
  },
  createdAt: number // ms epoch
}
```

`businesses/{businessId}`

One document per registered business. **The document ID is always the same as the owner's Firebase Auth UID** — this is a deliberate simplification: one business per owner account, and it means you never need to look up “which business does this owner have,” you already know it's `businesses/{ownerUid}`.

```
{
  id: string, // same as the doc ID / owner's uid
  name: string,
  ownerName: string,
  ownerUid: string,
  email: string,
  phone: string,
  category: string,
  address?: string,
  businessPhone?: string,
  whatsappNumber?: string,
  logoUrl?: string,
  plan: "free" | "standard" | "premium" | "premium_pro" | "enterprise",
}
```

```

planExpiry: number | null, // ms epoch; null = no active paid plan
walletBalance: number, // in NGN
currency: "NGN",
emailVerified: boolean,
onboarded: boolean, // has completed the post-signup profile step
createdAt: number
}

```

Subcollections of `businesses/{businessId}`

All of these live *inside* the business document, so one business's data never leaks into another's — the Firestore rule for this whole subtree is a single wildcard match (see Section 6).

`sales/{saleId}`

```

{
  items: string, // human-readable summary, e.g. "2x Rice, 1x Beans"
  lineItems: [{ name, qty, price }],
  total: number,
  customerName: string,
  customerPhone: string,
  customerEmail: string,
  status: "paid" | "owing",
  notes: string,
  createdAt: number
}

```

`purchases/{purchaseId}` — one document per line item (a single “log purchase” action with 3 items creates 3 documents)

```

{
  item: string,
  qty: number,
  cost: number, // unit cost
  supplier: string,
  total: number, // qty × cost
  createdAt: number
}

```

`inventory/{itemSlug}` — doc ID is a slugified version of the item name (see `lib/inventory.ts`), so the same item name from different sales/purchases always resolves to the same document

```

{
  name: string,
  quantity: number, // increased by purchases, decreased by sales — see Section 8
  unitCost: number, // most recent purchase cost
  updatedAt: number
}

```

`debtors/{debtorId}` — created automatically when a sale is logged with status `"owing"`

```

{
  customerName: string,
  customerPhone: string,
  customerEmail: string,
  balance: number,
  lastSaleAt: number
}

```

`customers/{customerId}` — doc ID is a slug of phone/email/name (see `lib/inventory.ts` → `slugifyItemName`). Upserted on every sale with any contact info, independent of debt status — this is the CRM data source.

```

{
  name: string,
  phone: string,
  email: string,
  totalSpent: number, // running total via Firestore `increment()`
  lastSaleAt: number
}

```

staff/{staffUid} — mirrors **users/{staffUid}** for easy display in the owner's Staff page

```
{
  name: string,
  email: string,
  permissions: { ...same shape as users/{uid}.permissions },
  createdAt: number
}
```

catalogue/{productId} — the owner's private product list (mirrored publicly, see **storefronts** below)

```
{
  name: string,
  price: number,
  imageUrl: string | null,
  createdAt: number
}
```

reviews/{reviewId} — screenshots of customer reviews, Premium feature

```
{
  customerName: string,
  imageUrl: string,
  createdAt: number
}
```

orders/{orderId} — reserved for future catalogue-order-placement flow (not currently written to by any page; the public storefront currently redirects orders to WhatsApp instead of writing an order document)

```
{
  customerName: string,
  total: number,
  status: string,
  createdAt: number
}
```

automation/config — single document, Review Automation settings (see Section 10)

```
{
  active: boolean,
  googleReviewLink: string,
  activatedAt: number | null
}
```

deposits/{depositId} **(top-level collection)**

Wallet top-up requests. Top-level (not a subcollection) so the admin console can query *across all businesses* with one query.

```
{
  businessId: string,
  businessName: string,
  amount: number,
  status: "pending" | "approved" | "rejected",
  requestedAt: number,
  resolvedAt?: number
}
```

withdrawals/{withdrawalId} **(top-level collection)**

Owner cash-out requests. Same cross-business-query reasoning as **deposits**.

```
{
  businessId: string,
  businessName: string,
  amount: number,
  bankName: string,
}
```

```

accountNumber: string,
accountName: string,
status: "pending" | "approved" | "rejected",
requestedAt: number,
resolvedAt?: number
}

```

`designRequests/{requestId}` (top-level collection)

Hire-a-Designer requests, ₦2,000 flat.

```

{
  businessId: string,
  businessName: string,
  brief: string,
  status: "pending" | "delivered",
  requestedAt: number,
  deliveredAt?: number
}

```

`messages/{messageId}` (top-level collection)

Owner ↔ Bamyon admin support chat. All businesses' messages live in one flat collection, filtered by `businessId` per conversation.

```

{
  businessId: string,
  businessName: string,
  sender: "owner" | "admin",
  text: string,
  createdAt: number,
  read: boolean
}

```

`storefronts/{businessId}` (top-level collection, publicly readable)

The **public-safe mirror** of a business's catalogue — this is what `/c/[businessId]` reads, so that the real `businesses/{id}` document (which has wallet balance, email, etc.) is never exposed to an unauthenticated visitor.

```

{
  name: string,
  whatsappNumber: string,
  logoUrl: string | null,
  address: string,
  updatedAt: number
}

```

Subcollection: `storefronts/{businessId}/products/{productId}` — same shape and same document ID as the corresponding `businesses/{id}/catalogue/{productId}` doc, written at the same time (see `components/pages/CatalogueManager.tsx`).

Storage structure summary (visual)

```

users/{uid}
businesses/{businessId} <- businessId === owner's uid
|-- sales/{saleId}
|-- purchases/{purchaseId}
|-- inventory/{itemSlug}
|-- debtors/{debtorId}
|-- customers/{customerId}
|-- staff/{staffUid}
|-- catalogue/{productId}
|-- reviews/{reviewId}
|-- orders/{orderId}
`-- automation/config
deposits/{depositId}
withdrawals/{withdrawalId}
designRequests/{requestId}
messages/{messageId}

```

```
storefronts/{businessId}
  -- products/{productId}
```

4. Firebase Authentication

Only **Email/Password** sign-in is enabled — no OAuth providers, no phone auth. Three account types share the same Firebase Auth user pool; what distinguishes them is purely the `role` field on their `users/{uid}` document, not anything in Firebase Auth itself.

- **Owner:** self-registers via `/signup`. Gets a `users/{uid}` doc with `role: "owner"` and a matching `businesses/{uid}` doc, created together in one atomic `writeBatch` (see Section 9).
- **Staff:** created *by* an owner, from `/dashboard/staff`, using a workaround described in Section 7 (since there's no Admin SDK to create a user without switching the current session).
- **Admin:** no public signup exists. You create one by signing up normally, then manually editing that `users/{uid}` document's `role` field to `"admin"` in the Firebase console.

Email verification uses Firebase's native link-based flow (`sendEmailVerification`) — not a custom 6-digit code — specifically because it works without standing up any transactional email infrastructure of your own.

5. Cloudinary — image uploads

Used in three places: business logo (Settings), review screenshots (Reviews), catalogue product photos (Catalogue).

Setup: 1. Free account at cloudinary.com. 2. Copy the **Cloud Name** from the dashboard. 3. Settings → Upload → Add upload preset → **Signing Mode: Unsigned** → save, copy the preset name. 4. Both values go into `.env.local` as `NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME` and `NEXT_PUBLIC_CLOUDINARY_UPLOAD_PRESET`.

Why unsigned uploads: an unsigned preset lets the browser upload directly to Cloudinary's API with no secret key involved — no backend upload handler needed, and nothing sensitive is exposed. This is the entire implementation, in `lib/cloudinary.ts`:

```
const form = new FormData();
form.append("file", file);
form.append("upload_preset", UPLOAD_PRESET);

const res = await fetch(
  `https://api.cloudinary.com/v1_1/${CLOUD_NAME}/image/upload`,
  { method: "POST", body: form }
);
```

`components/ImageUpload.tsx` wraps this with a file picker, a preview thumbnail, and toast feedback.

`components/ReceiptScanner.tsx` does its own separate base64 conversion (not through Cloudinary) because the AI vision endpoint needs a data URL, not a hosted URL.

6. How security works without a backend server

This is the part that most needs to be understood before extending the app, not just read once.

The problem: normally, “spend ₦2,500 from a wallet and grant a plan” would be server-side logic you trust. Here, the browser itself calls Firestore's `runTransaction` directly. Nothing stops a technically savvy user from opening devtools and trying to write `walletBalance: 999999999` straight into their own business document.

The fix is entirely in `firestore.rules`. The key insight: a wallet balance can only ever be exploited by *increasing* it without paying, or by *decreasing* it by less than a plan actually costs while still granting that plan. Both are preventable with rules alone:

```
allow update: if isAdmin() || (
  isOwnerOf(businessId) && (
    // Case A: ordinary profile edit – financial fields untouched
```

```

!diff.affectedKeys().hasAny(['plan','planExpiry','walletBalance','ownerUid'])
||
// Case B1: plan upgrade – plan changes, balance must drop by an EXACT
// known price (hardcoded list in the rules file)
(diff.affectedKeys().hasOnly(['plan','planExpiry','walletBalance']) &&
newBalance < oldBalance && newBalance >= 0 &&
(oldBalance - newBalance) in validPlanPrices())
||
// Case B2: wallet-only spend (design request, automation activation,
// withdrawal) – plan/planExpiry untouched, so ANY decrease is safe:
// spending your own balance can never create value out of nothing
(diff.affectedKeys().hasOnly(['walletBalance']) &&
newBalance < oldBalance && newBalance >= 0)
)
);

```

The only way `walletBalance` ever *increases* is `isAdmin()` — a real, separately-authenticated Firebase Auth session with `role: "admin"`, resolved via approving a deposit or rejecting a withdrawal (which refunds it). No client trick reaches that branch.

Three specific workarounds for not having an Admin SDK:

1. **Staff account creation** (`lib/staffActions.ts`): creating a Firebase Auth user with the client SDK normally signs you in as that new user — which would kick the owner out of their own session. Fixed by spinning up a second, throwaway Firebase App instance (`initializeApp(config, "StaffCreation")`) just to create the account, then discarding it. The owner's real session, on the default app instance, never moves.
2. **CRM email campaigns** (`app/api/crm/send-campaign/route.ts`): this is the one feature that genuinely needs a server, because a real secret (`RESEND_API_KEY`) can't live in the browser. It's still not using the Admin SDK, though — it verifies the caller's Firebase ID token by checking its cryptographic signature against **Google's public keys** (`lib/verifyIdToken.ts`, via the `jose` package), which requires no credentials at all, then reads Firestore over its public REST API using that same ID token as the auth header (`lib/firestoreRest.ts`) — so the read is subject to your normal Firestore rules, exactly as if the owner had made it from the browser.
3. **Design requests** (residual limitation, documented honestly): the wallet deduction and the `designRequests` document are written in the same client-side `runTransaction`, but Firestore rules can't *fully* guarantee two separate documents change together the way a real server transaction would. The rule requires the business to have had $\geq \$2,000$ at request time as a second check, which closes most of the gap. A determined attacker bypassing the app's UI entirely could theoretically create a request without the paired deduction. Low real-world risk; if it ever matters, move just that one write to a small serverless function.

7. Staff accounts and permissions

An owner creates a staff login from `/dashboard/staff`: name, email, temporary password, and a set of checkboxes — one per permission key (`sales`, `purchases`, `inventory`, `debtors`, `receipts`, `invoices`, `catalogue`, `crm`, `orders`, `reviews`, `analytics`).

Staff log in at the same `/login` page as owners, but land in a **completely separate interface** at `/staff` — not the owner dashboard with menu items hidden. This matters: `/staff/*` and `/dashboard/*` are two distinct route trees (`app/staff/layout.tsx` vs `app/dashboard/layout.tsx`), each redirecting the wrong role away if they land there directly.

Every staff page is wrapped in `<StaffGate permission="..." minPlan="...">` (`components/StaffGate.tsx`), which checks **two things**, not one: 1. Does `userDoc.permissions[permission] === true`? (did the owner grant this) 2. Does the business's current plan unlock this feature? (`planUnlocks()` in `lib/permissions.ts`)

A staff member can't use a feature the business hasn't paid for, even if the owner ticked the box for it — the plan check is independent of the permission check.

Staff-account limits by plan (enforced in `lib/staffActions.ts`): 0 on Free, 1 on Standard, unlimited on Premium and above.

8. Automatic inventory tracking

`lib/inventory.ts` exports two functions used by `SalesForm` and `PurchasesForm` :

- `increaseInventory(businessId, name, qty, unitCost)` — called for every line item on a logged purchase. Upserts `businesses/{id}/inventory/{slug(name)}` with `quantity: increment(qty)` and refreshes `unitCost` to the latest purchase price.
- `decreaseInventory(businessId, name, qty)` — called for every line item on a logged sale. Same document, `quantity: increment(-qty)`.

Item names are matched by a **slug** (`slugifyItemName` : lowercase, non-alphanumeric stripped to hyphens), so “Rice”, “rice”, and “RICE” all resolve to the same inventory document. This is what powers the **Net Worth tracker**:

`inventoryValue =  $\Sigma(\max(0, \text{quantity}) \times \text{unitCost})$` , plus wallet balance.

9. Signup and onboarding — why they’re built the way they are

This deserves explanation because an earlier version had a real bug here that’s worth understanding so it doesn’t come back.

Signup (`app/signup/page.tsx`) creates the Firebase Auth account, then writes `businesses/{uid}` and `users/{uid}` in **one atomic writeBatch** , awaited before navigating anywhere. This used to be two separate unawaited writes with silently swallowed errors — if either lost the race or failed, the person would land on the next screen with no account data and no indication anything was wrong. The batch write makes that class of bug structurally impossible: either both documents exist or neither does.

Onboarding (`app/onboarding/page.tsx`) uses `setDoc(..., { merge: true })` , not `updateDoc` — `updateDoc` throws if the target document doesn’t exist yet, which is exactly the failure mode a signup race could produce. `setDoc` with merge can’t fail that way.

Login (`app/login/page.tsx`) resumes exactly where an owner left off, not just “always go to dashboard”: unverified email → `/verify-email` , verified but `business.onboarded === false` → `/onboarding` , otherwise → `/dashboard` . This costs one extra Firestore read, but only for owners — staff and admin skip straight to their own interface.

Every Firestore listener in the app goes through `lib/firestoreListen.ts` (`listenWithErrorToast`) , not a bare `onSnapshot` — it attaches an error callback that shows a toast and logs to console. Without this, a missing composite index or a rules rejection fails completely silently and the page spins forever. `contexts/ToastContext.tsx` is the toast system every write and read error surfaces through.

10. Free AI integrations (Pollinations.ai)

Three separate uses of the same free, keyless service:

Text rewriting (`lib/pollinationsText.ts`) — a GET request to `https://text.pollinations.ai/{urlencoded-prompt}` returns plain text back. Used for the admin’s “Amplify” button on support replies, and the CRM page’s “Improve with AI” for campaign messages.

Image generation (`lib/pollinations.ts`) — a GET request to `https://image.pollinations.ai/prompt/{urlencoded-prompt}?width=512&height=512&nologo=true&seed={n}` returns an image directly, usable straight as an `<img src>` . Powers the free AI logo generator in Settings.

Vision (receipt scanning, Premium) (`lib/pollinationsVision.ts`) — a POST to `https://text.pollinations.ai/openai` in OpenAI-chat-completion format, with an image content block (`{ type: "image_url", image_url: { url: base64DataUrl } }`) , asking for strict JSON back describing line items. The response is defensively parsed (regex-extract the JSON block, catch parse failures) and **never auto-saved** — it only pre-fills the Log Sale / Log Purchase form for a human to review, because receipt OCR is never perfect.

None of these three require an API key, account, or billing setup.

11. Resend — Premium Pro email campaigns

`app/api/crm/send-campaign/route.ts` loops over selected customers (anyone in `businesses/{id}/customers` with an email on file) and calls Resend's REST API directly:

```
await fetch("https://api.resend.com/emails", {
  method: "POST",
  headers: { Authorization: `Bearer ${apiKey}`, "Content-Type": "application/json" },
  body: JSON.stringify({ from: fromAddress, to: recipient.email, subject, text }),
});
```

Setup: free account at resend.com, verify a sending domain (or use their test domain for development), create an API key, set `RESEND_API_KEY` and `RESEND_FROM_EMAIL` in environment variables.

Important framing point, worth restating: this sends from a verified address via Resend — it is *not* literally sending through the business owner's personal Gmail account. True “send as the owner's Gmail” would require Google OAuth consent per business owner, a materially bigger and riskier integration. This is the credible, deliverable version of “one-click email to past customers.”

12. n8n — Review Automation

An owner activates this for a flat ₦1,000 (from wallet balance) and pastes their Google Business Profile review link. From then on, every sale logged with a customer phone number triggers `app/api/automation/notify/route.ts`, which forwards a payload to an n8n webhook URL (kept server-side in `N8N_WEBHOOK_URL`, never exposed to the browser):

```
{
  "businessName": "...",
  "customerName": "...",
  "customerPhone": "...",
  "googleReviewLink": "...",
  "saleTotal": 4500,
  "triggeredAt": 1234567890
}
```

Bamyon's responsibility ends at firing this webhook. The actual SMS/WhatsApp send is a workflow you build in n8n: a Webhook trigger node, then whatever SMS/WhatsApp provider node you choose. This call is fire-and-forget and wrapped in try/catch on both ends — a failure here never blocks or fails the underlying sale.

Admin can see every business that has activated this at `/admin/automation`, via a Firestore `collectionGroup` query across every `businesses/*/automation/config` document.

13. Plan tiers and pricing

Defined entirely in `lib/types.ts`:

```
PLAN_PRICES = {
  standard: { monthly: 2500, "6months": 13500, yearly: 25000 },
  premium: { monthly: 5000, "6months": 27000, yearly: 50000 },
  premium_pro: { monthly: 15000, "6months": 81000, yearly: 150000 },
}
```

`enterprise` has no fixed price — it's a “Contact Sales” card that routes to the support chat, not part of the self-serve wallet flow.

Feature gating is rank-based, not a hardcoded list of exceptions: `lib/permissions.ts` assigns a numeric rank to each tier (free: 0 through enterprise: 4) and `planUnlocks(business, minPlan)` just compares ranks. Every nav item and every `<PlanGate minPlan="...">` wrapper uses this same function, so Premium Pro and Enterprise automatically also unlock everything Standard/Premium unlock — there's no duplication of “who gets what.”

14. The wallet, end to end

Every money-related feature spends from or credits the same `businesses/{id}.walletBalance` field:

1. **Fund the wallet:** owner sees Bamyon's bank account on `/dashboard/wallet`, transfers money manually, taps "I have transferred," logs the amount → creates a `pending` doc in `deposits`.
2. **Admin credits it:** `/admin/payments`, tap Approve → `lib/walletAdminActions.ts` runs a client-side transaction (from the admin's own authenticated session) that marks the deposit approved and increases `walletBalance`.
3. **Spend it:** four different features all decrease the same field, via `runTransaction`, validated by the rules in Section 6 — plan upgrades (`lib/upgradeActions.ts`), Hire-a-Designer (`lib/designActions.ts`), Review Automation activation (`lib/automationActions.ts`), and withdrawal requests (`lib/withdrawalActions.ts`, which *holds* the balance immediately on request rather than waiting for admin approval).
4. **Withdraw it:** owner requests a payout with bank details; balance is held immediately; admin pays out manually via real bank transfer, then marks it approved (or rejected, which refunds the held balance) at `/admin/withdrawals`.

15. Recreating this project from scratch — step by step

If you were starting over with nothing but this document:

1. **Scaffold:** `npx create-next-app@14` with TypeScript, Tailwind, App Router.
2. **Firebase project:** `console.firebase.google.com` → new project → enable Firestore (production mode) → Authentication → Email/Password provider.
3. **Get the client config:** Project Settings → General → Add app → Web → copy the config object into `.env.local` as the six `NEXT_PUBLIC_FIREBASE_*` variables. This is the *only* Firebase credential needed anywhere in this project.
4. **Write `lib/firebase.ts`:** initialize the client SDK (`initializeApp`, `getAuth`, `getFirestore`), export `db`, `auth`, and the raw `firebaseConfig` object (the last one is needed later for the staff-creation workaround).
5. **Write `lib/types.ts`** first, before any UI: every interface in Section 3 above, plus the plan pricing/feature constants from Section 13. Getting the data model right before writing pages saves rework.
6. **Build `contexts/ToastContext.tsx` and `contexts/AuthContext.tsx`** before any real page — every other piece of the app depends on `useAuth()` (current Firebase user, their `users/{uid}` doc, their `businesses/{id}` doc) and `useToast()`. Wrap the root layout: `<ToastProvider><AuthProvider>{children}</AuthProvider></ToastProvider>` — that order matters, since `AuthProvider` itself calls `useToast()` to surface listener errors.
7. **Write `firestore.rules`** from Section 6 before writing any page that touches money — build the rules and the client code that depends on them together, not the rules as an afterthought.
8. **Auth pages:** `/signup` (atomic `writeBatch`, Section 9), `/login` (role + onboarding-status routing), `/verify-email` (polling `reload()` every few seconds), `/onboarding`, `/forgot-password`.
9. **Owner dashboard:** `app/dashboard/layout.tsx` (route guard + onboarding redirect + stuck-state timeout fallback), then each page under `app/dashboard/*` — start with Sales and Purchases since Net Worth and Analytics depend on their data existing.
10. **Staff dashboard:** extract each dashboard page's actual content into `components/pages/*.tsx` so both `/dashboard/*` and `/staff/*` can import the same component, wrapped differently (`<PlanGate>` for owners, `<StaffGate>` for staff). Build `lib/staffActions.ts` (the secondary-Firebase-app trick) before the Staff management page.
11. **Admin console:** `app/admin/layout.tsx`, then `Businesses/Payments/Withdrawals/Design Requests/Automation/Messages` pages. No public signup — document the manual role-flip process (Section 4).
12. **Cloudinary:** `lib/cloudinary.ts` + `components/ImageUpload.tsx`, then wire into `Settings/Reviews/Catalogue`.
13. **AI integrations:** `lib/pollinations*.ts` files (Section 10) — these have no setup dependency, build them any time.
14. **Resend + n8n:** build these last, since they're the two features that need real external accounts and secrets before they can be tested end to end.
15. **Public storefront:** `storefronts` collection + `/c/[businessId]/page.tsx` — build after Catalogue, since it mirrors that data.
16. **Deploy:** push to GitHub, import into Vercel, add every `.env.local` variable to Vercel's environment variables, deploy. Then deploy `firestore.rules` and `firestore.indexes.json` from the Firebase console or CLI.

16. Environment variables reference

Firebase client SDK — the only Firebase credential needed anywhere

```
NEXT_PUBLIC_FIREBASE_API_KEY=  
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=  
NEXT_PUBLIC_FIREBASE_PROJECT_ID=  
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=  
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=  
NEXT_PUBLIC_FIREBASE_APP_ID=  
  
# Shown on the Wallet page  
NEXT_PUBLIC_BANK_NAME=  
NEXT_PUBLIC_BANK_ACCOUNT_NUMBER=  
NEXT_PUBLIC_BANK_ACCOUNT_NAME=  
  
# Cloudinary – unsigned preset, no secret key  
NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME=  
NEXT_PUBLIC_CLOUDINARY_UPLOAD_PRESET=  
  
# Resend – Premium Pro email campaigns (server-only secret)  
RESEND_API_KEY=  
RESEND_FROM_EMAIL=  
  
# n8n – Review Automation webhook (server-only, never exposed to the browser)  
N8N_WEBHOOK_URL=
```

Everything prefixed `NEXT_PUBLIC_` is bundled into the browser and is not a secret by design (Firebase’s real security boundary is `firestore.rules`, not hiding the config). `RESEND_API_KEY` and `N8N_WEBHOOK_URL` are the only two values in this project that are actual secrets and must stay server-side only.

17. Known limitations (be honest about these before extending)

- **No automated plan-expiry downgrade.** `planExpiry` is checked client-side for *display*, but nothing currently flips an expired business back to `free` server-side. Needs a scheduled job (Vercel Cron hitting a small API route, checking `planExpiry < now` across all businesses).
- **Design request atomicity gap** — see Section 6, point 3.
- `orders` **subcollection is defined but unused** — the public storefront currently sends orders straight to WhatsApp instead of writing an order document. Wiring that up is a natural next step for the Orders page to have real data.
- **Review Automation’s actual SMS/WhatsApp sending lives entirely in n8n**, outside this codebase — Bamyon only fires the trigger.
- **AI vision scanning accuracy is only as good as Pollinations’ free vision model** on a given day; it’s explicitly a pre-fill-and-review feature, never a silent auto-save, for exactly this reason.

This document reflects the codebase as of v5. Cross-reference `README.md` in the project root for setup commands and deployment steps specifically — this file is the “how and why,” that one is the “run these commands.”